Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

ОТЧЁТ
по лабораторной работе №3
по дисциплине
«Информационные технологии и веб-программирование»
Тема: Реализация системы аутентификации и авторизации

Выполнила: студентка группы 320604
Хаджинова К.А.

Проверил: Хаджинова Н.В.

Минск 2026

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ 3
2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 3
3 ХОД РАБОТЫ 3
3.1 Модель пользователя и миграция 3
3.2 Регистрация и вход 4
3.3 Промежуточный обработчик и защищённые маршруты 7
3.4 Дополнительный механизм: роли 8
3.5 Проверка 11
3.6 Ограничения 12
3.7 Постраничный вывод и сортировка 12
3.8 Метод PATCH 13
3.9 Как вызвать ответ 500 настоящим сбоем 13
3.10 Ответы на контрольные вопросы 15
3.11 Архитектура и компромиссы 16
3.12 База данных 16
ЗАКЛЮЧЕНИЕ 17
ПРИЛОЖЕНИЕ А Листинг файла user.js 18
ПРИЛОЖЕНИЕ Б Листинг миграции создания таблицы Users 19
ПРИЛОЖЕНИЕ В Листинг файла auth.js 21
ПРИЛОЖЕНИЕ Г Листинг файла auth.controller.js 22
ПРИЛОЖЕНИЕ Д Листинг файла validate-auth.js 24
ПРИЛОЖЕНИЕ Е Листинг файла users.routes.js 26
ПРИЛОЖЕНИЕ Ж Листинг файла diagrams.controller.js 27
ПРИЛОЖЕНИЕ И Листинг файла list-query.js 29
ПРИЛОЖЕНИЕ К Листинг файла validate.js 31

1 ЦЕЛЬ РАБОТЫ

Освоить механизмы аутентификации и авторизации в веб-приложениях: регистрацию и вход пользователей с токенами JWT, хеширование паролей с помощью bcrypt и защиту маршрутов промежуточным обработчиком. Работа продолжает лабораторные работы 1 и 2 для проекта DSLKit, веб-редактора диаграмм как кода, тематика которого соответствует направлению SaaS. В качестве дополнительного механизма безопасности выбрана ролевая модель RBAC.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Аутентификация и авторизация. Аутентификация отвечает на вопрос, кто делает запрос, авторизация на вопрос, что этому пользователю разрешено. Неверные учётные данные дают ответ 401, а нехватка прав у известного пользователя ответ 403.

JWT. Токен состоит из трёх частей, разделённых точками: заголовок с алгоритмом, полезная нагрузка с утверждениями и подпись. Подпись вычисляется секретным ключом сервера, поэтому изменить нагрузку незаметно нельзя, а проверить подлинность может только владелец ключа. Нагрузка не шифруется, поэтому пароли и другие секреты в неё не кладут. Срок жизни задаётся утверждением `exp`, и по его истечении токен отклоняется.

Хеширование паролей. Пароль хранят только в виде хеша. Алгоритм bcrypt добавляет к каждому паролю случайную соль и намеренно работает медленно, что делает подбор по украденной базе дорогим. Параметр стоимости 10 означает 2 в степени 10 итераций. Алгоритм учитывает только первые 72 байта пароля.

Bearer-токен. Клиент передаёт токен в заголовке `Authorization: Bearer <токен>`. Сервер проверяет подпись и срок, а затем решает, пускать ли запрос к маршруту.

3 ХОД РАБОТЫ

3.1 Модель пользователя и миграция

Таблица `Users` создана третьей миграцией: `id`, `email` с уникальным индексом, `passwordHash`, `role` со значением по умолчанию `user` и служебные поля Sequelize. Модель `User` проверяет роль списком `user` и `admin` и имеет метод `toPublic`, который возвращает пользователя без хеша. Все ответы строятся через этот метод, поэтому хеш не может попасть в ответ

по недосмотру. Пакеты `bcrypt` и `jsonwebtoken` установлены командой `npm install`.

3.2 Регистрация и вход

Маршрут `POST /auth/register` приводит email к нижнему регистру, проверяет его формат и пароль, хеширует пароль вызовом `bcrypt.hash` и сохраняет пользователя. Ответ 201 содержит `id`, `email`, `role` и дату создания. Уникальность обеспечивает индекс базы: нарушение превращается в ответ 409, в том числе при одновременных запросах с одним email. Роль при регистрации всегда `user`, поле `role` из тела запроса игнорируется. Пароль должен содержать от 8 символов и не больше 72 байт, иначе bcrypt молча обрезал бы его.

Маршрут `POST /auth/login` находит пользователя, сравнивает пароль с хешем и выдаёт токен, подписанный алгоритмом HS256, со сроком жизни из переменной `JWT_EXPIRES_IN`, по умолчанию один час. В нагрузку входят `id`, `email` и `role`. Для неверного пароля и для неизвестного email сервер возвращает одинаковый ответ 401, а при отсутствии пользователя сравнивает пароль с запасным хешем, чтобы время ответа не выдавало существование учётной записи. Секрет `JWT_SECRET` обязателен и должен содержать не меньше 32 символов, иначе сервер не запускается. В репозитории лежит только образец `.env.example`.

Миграции и сид выполнены командами `npm run db:migrate` и `npm run db:seed`, первый администратор взят из переменных окружения, созданных при запуске. Вывод показан на рисунке 1.



Рисунок 1 – Вывод миграций и сида в консоли (вывод процессов, снятый как текст)

Сервер запущен командой `npm start`, сообщение консоли показано на рисунке 2.

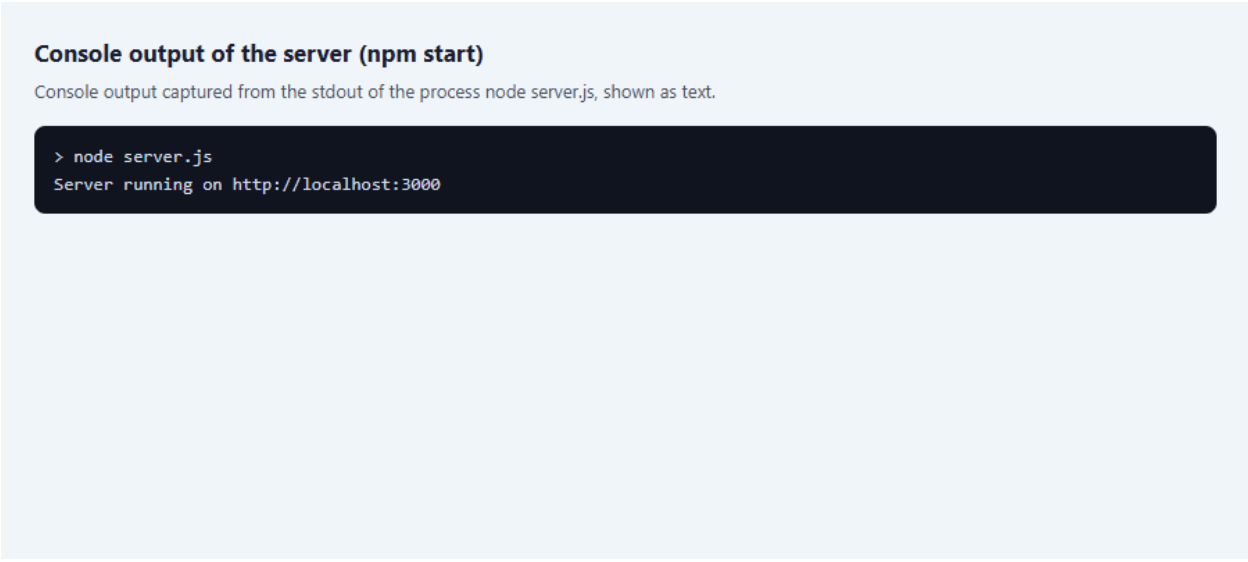


Рисунок 2 – Сообщение консоли после запуска сервера на порту 3000 (вывод процесса, снятый как текст)

Браузер не отправляет запросы `POST`, `PUT`, `PATCH` и `DELETE` из адресной строки, поэтому такие запросы отправил скрипт `shots/shots.mjs`, а настоящие запросы и ответы сервера выведены на странице. Регистрация, отказ при повторе адреса и отказ при слабом пароле показаны на рисунке 3.



Registration

HTTP exchange recorded by a script (fetch). The answers are real answers of the running server, the JSON body is only indented, a token is cut in the Authorization line.

REGISTER

```
POST /auth/register HTTP/1.1
Host: localhost:3000
Content-Type: application/json

{
  "email": "kate@example.com",
  "password": "correct horse 42"
}
```

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Location: /profile

{
  "id": 2,
  "email": "kate@example.com",
  "role": "user",
  "createdAt": "2026-10-21T06:31:04.492Z"
}
```

THE SAME EMAIL IN ANOTHER LETTER CASE

```
POST /auth/register HTTP/1.1
Host: localhost:3000
Content-Type: application/json

{
  "email": "Kate@example.com",
  "password": "correct horse 42"
}
```

```
HTTP/1.1 409 Conflict
Content-Type: application/json; charset=utf-8

{
  "error": "A user with this email already exists"
}
```

A SHORT PASSWORD

```
POST /auth/register HTTP/1.1
Host: localhost:3000
Content-Type: application/json

{
  "email": "bob@example.com",
  "password": "short"
}
```

```
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8

{
  "error": "Invalid credentials data",
  "details": [
    {
      "field": "password",
      "message": "must be at least 8 characters"
    }
  ]
}
```

Рисунок 3 – Запросы регистрации и ответы 201, 409 и 400, записанные скриптом (не окно Postman)

Вход с верным и неверным паролем показан на рисунке 4.



Login

HTTP exchange recorded by a script (fetch). The answers are real answers of the running server, the JSON body is only indented, a token is cut in the Authorization line.

LOG IN

```
POST /auth/login HTTP/1.1
Host: localhost:3000
Content-Type: application/json

{
  "email": "kate@example.com",
  "password": "correct horse 42"
}
```

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MiwiZW1haWwiOiJrYXRlQGV4YW1wbGUuY29tIiwicm9sZSI6InVzZXIiLCJpYXQiOjE3OTI1NjQyNjUsImV4cCI6MTc5MjU2Nzg2NSwiaXNzIjoiZHNsa2l0Iiwic3ViIjoiMiJ9.-9GSssnaFF0hfsrQ9yIv5ygNbE8wFum0AxABufTqnVw",
  "tokenType": "Bearer",
  "expiresIn": "1h"
}
```

WRONG PASSWORD

```
POST /auth/login HTTP/1.1
Host: localhost:3000
Content-Type: application/json

{
  "email": "kate@example.com",
  "password": "wrong password"
}
```

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json; charset=utf-8

{
  "error": "Invalid email or password"
}
```

Рисунок 4 – Запросы входа: ответ 200 с токеном и ответ 401 при неверном пароле, записанные скриптом (не окно Postman)

3.3 Промежуточный обработчик и защищённые маршруты

Обработчик `authenticate` читает заголовок `Authorization`, проверяет схему `Bearer`, проверяет подпись и срок токена и принимает только алгоритм HS256, поэтому токены с алгоритмом `none` или HS512 отклоняются. Затем он читает пользователя из базы и кладёт в `req.user` значения `id`, `email` и `role`. Чтение из базы выбрано сознательно: удалённая учётная запись и изменённая роль начинают действовать сразу, а не после истечения старого токена. Цена решения один дополнительный запрос на каждый защищённый вызов.

Защищены маршрут `GET /profile`, возвращающий текущего пользователя, и все маршруты `/diagrams`. Результаты проверки без токена и с неверным токеном перечислены в таблице 1.

Таблица 1 – Ответы защищённых маршрутов при разных токенах

Токен	**Код**	**Сообщение**
Нет заголовка, чужая схема, лишние слова	401	Missing Bearer token
Изменена нагрузка, чужая подпись, алгоритм none или HS512	401	Invalid token
Срок жизни истёк	401	Token expired
Пользователь удалён	401	User no longer exists
Верный токен	200	данные запроса

Доступ к профилю с верным токеном, без токена и с изменённым токеном показан на рисунке 5.

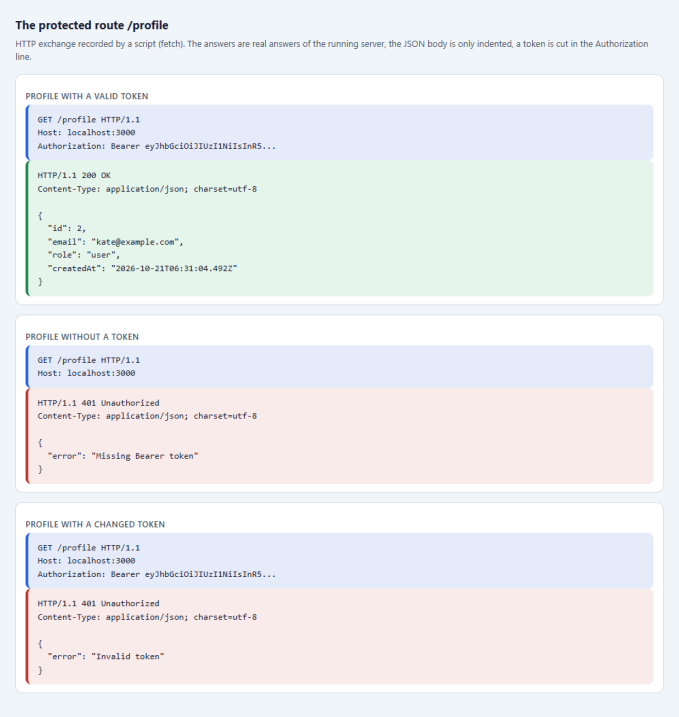


Рисунок 5 – Запросы GET /profile: ответ 200 с верным токеном и ответы 401 без токена и с изменённым токеном, записанные скриптом (не окно Postman)

Адрес `http://localhost:3000/profile`, открытый в браузере без заголовка `Authorization`, даёт ответ, показанный на рисунке 6.



Рисунок 6 – Ответ GET /profile в окне браузера без токена: JSON с сообщением об отсутствии токена (код 401)

3.4 Дополнительный механизм: роли

Выбрана ролевая модель, потому что в SaaS-редакторе нужны администратор и обычный пользователь. Middleware `requireRole('admin')` возвращает 403 всем, кроме администраторов. Администратору доступны три маршрута, перечисленные в таблице 2. Кроме того, диаграмму может менять

и удалять только её владелец или администратор: владельцем становится вызывающий пользователь, а поле `ownerId` в теле запроса игнорируется. Читать диаграммы может любой вошедший пользователь.

Таблица 2 – Маршруты и права доступа

Маршрут	**Кто может**	**Ответы**
`POST /auth/register`, `POST /auth/login`	все	201 или 200, 400, 401, 409
`GET /profile`	вошедший пользователь	200, 401
`GET /diagrams`, `GET /diagrams/:id`, `POST /diagrams`	вошедший пользователь	200 или 201, 400, 401, 404
`PUT /diagrams/:id`, `DELETE /diagrams/:id`	владелец или admin	200 или 204, 403, 404
`GET /users`	admin	200, 401, 403
`PATCH /users/:id/role`	admin	200, 400, 404
`DELETE /users/:id`	admin	204, 400, 404

Администратор не может изменить собственную роль или удалить себя, чтобы система не осталась без администраторов. Первый администратор создаётся сидом из переменных `ADMIN_EMAIL` и `ADMIN_PASSWORD`, поэтому пароль не хранится в репозитории. Если переменные не заданы, сид пропускается.

Создание диаграммы вошедшим пользователем, частичное изменение и отказ при пустом теле показаны на рисунке 7.



Рисунок 7 – Запросы POST и PATCH с токеном владельца и ответы 201, 200 и 400, записанные скриптом (не окно Postman)

Права владельца, постороннего пользователя и администратора показаны на рисунке 8.



Рисунок 8 – Ответы 403 постороннему пользователю и ответ 200 администратору, записанные скриптом (не окно Postman)

3.5 Проверка

Работа проверена автоматическими тестами на временной базе SQLite с настоящими миграциями и настоящими HTTP-запросами. Тесты запускают сервер на свободном порту, а bcrypt использует четыре раунда для скорости. Покрыто следующее: регистрация с нормализацией email, отсутствие хеша в ответе и проверка, что в базе лежит хеш bcrypt, а не пароль, дубликаты email в любом регистре, ошибки проверки данных, вход и содержимое токена, одинаковый ответ на неверный пароль и неизвестный email, подделанные, неподписанные, просроченные и подписанные чужим ключом токены, удаление пользователя и смена роли при старом токене, права администратора, владельца и постороннего пользователя, миграции с откатом и сид администратора. Выполнено 118 тестов приложения, включая частичное обновление `PATCH` и сбой базы. Отдельно 28 октября 2026 выполнен прогон на настоящем PostgreSQL 16 в контейнере: миграции, сид администратора, регистрация, вход, защищённый маршрут с токеном и без него и проверка ролей отработали так же, как на SQLite. Снимки лежат в папке `figures-pg`.

Результат команды `npm test` с числом тестов и покрытием кода показан на рисунке 9.

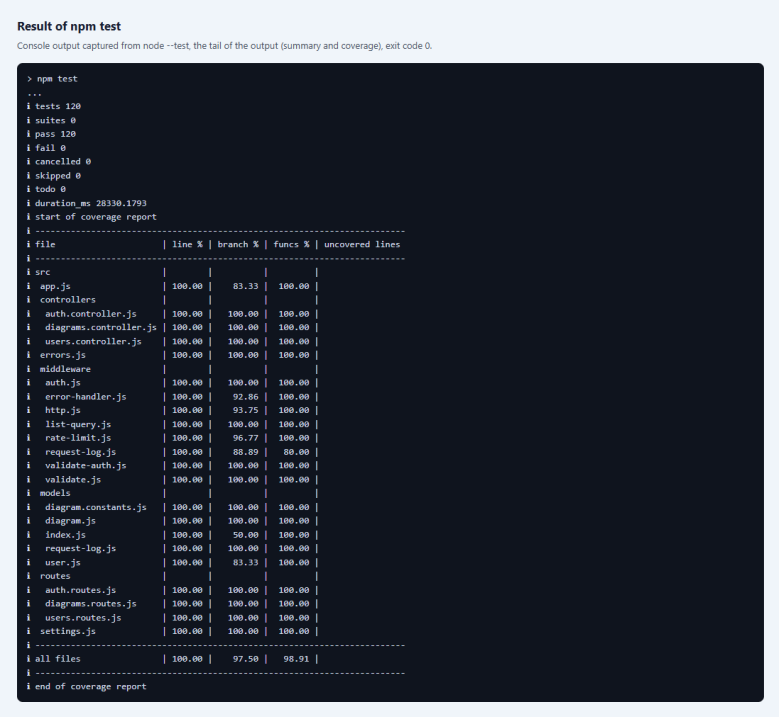


Рисунок 9 – Конец вывода команды npm test: число пройденных тестов и таблица покрытия (вывод процесса, снятый как текст)

3.6 Ограничения

Выход из системы не отзывает токен: он действует до истечения срока, поэтому срок выбран коротким. Число попыток входа ограничено: не более 20 запросов входа с одного адреса за 15 минут, а после 5 неудачных попыток почта блокируется на 15 минут. Счётчики хранятся в памяти процесса. Токен хранится на стороне клиента, и выбор между localStorage и cookie с флагом httpOnly делается в клиентской лабораторной работе.

Листинг файла user.js приведён в приложении А. Листинг миграции создания таблицы Users приведён в приложении Б. Листинг файла auth.js приведён в приложении В. Листинг файла auth.controller.js приведён в приложении Г. Листинг файла validate-auth.js приведён в приложении Д. Листинг файла users.routes.js приведён в приложении Е. Листинг файла diagrams.controller.js приведён в приложении Ж. Листинг файла validate.js приведён в приложении К.

3.7 Постраничный вывод и сортировка

Список принимает параметры `limit`, `offset`, `sort` и `order`. Параметр `limit` задаёт размер страницы от 1 до 100, `offset` задаёт смещение от 0 до 1000000, `sort` выбирает поле из белого списка, а `order` принимает значения `asc` и `desc`. Без параметров возвращается весь список по возрастанию `id`, поэтому клиенты прежних работ продолжают работать. Ответ остаётся массивом, а полный размер выборки приходит в заголовке `X-Total-Count`. Числа читаются как строки не длиннее девяти цифр, поэтому значения `-1`, `1e9`, `0x10` и огромные числа получают ответ 400 со списком ошибок по полям. Имя поля сортировки проверяется по белому списку и не попадает в запрос к базе без проверки. Равные значения упорядочены по `id`, поэтому страницы не пересекаются. Разбор параметров вынесен в отдельный модуль, он приведён в приложении И. Параметр `notation` оставляет записи одной нотации, а параметр `q` ищет часть названия без учёта регистра функцией `instr` в SQLite или `strpos` в PostgreSQL. Недопустимое значение даёт ответ 400. Число неудачных входов до блокировки, срок блокировки и пределы запросов на вход и регистрацию лежат в файле `limits.json`, а не в коде.

Запрос `GET /diagrams?limit=2&offset=0&sort=title&order=desc` с токеном возвращает страницу из двух записей, как на рисунке 10.

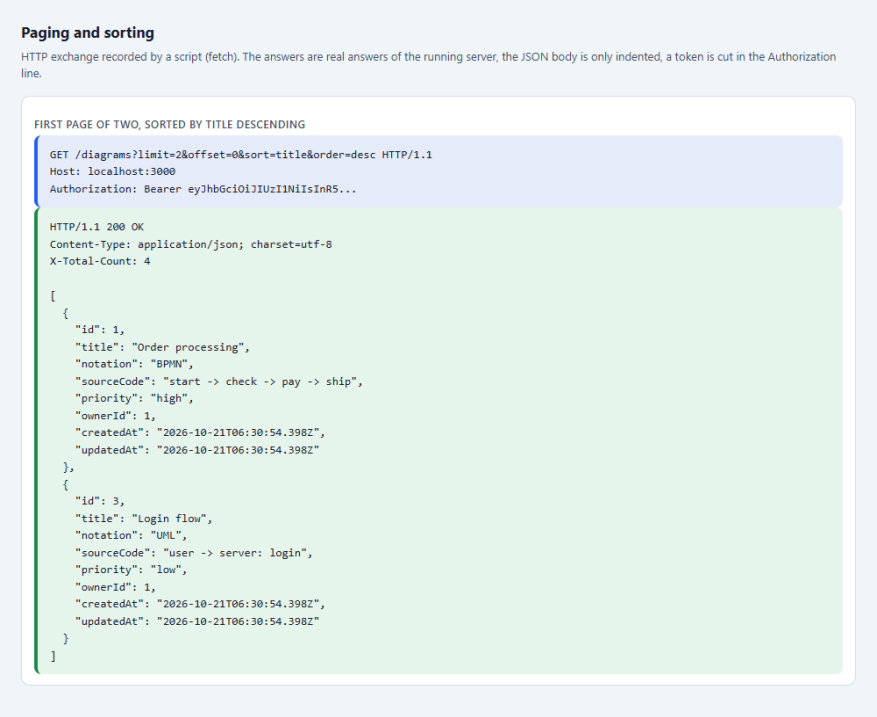


Рисунок 10 – Запрос списка с параметрами limit, offset, sort и order, записанные скриптом (не окно Postman)

3.8 Метод PATCH

Маршрут `PATCH /diagrams/:id` меняет только присланные поля. Проверка вынесена в `validatePartial`, она приведена в приложении К. Каждое присланное поле проходит то же правило, что в `POST` и `PUT`, тело без единого известного поля даёт 400, а поле владельца прислать нельзя. Права такие же, как у `PUT`: менять диаграмму может владелец или администратор, остальные получают 403. Метод `PUT` заменяет запись целиком и требует все обязательные поля, а `PATCH` оставляет пропущенные поля как есть. Поэтому `PUT` идемпотентен, а `PATCH` по определению не обязан быть таким.

3.9 Как вызвать ответ 500 настоящим сбоем

Ответ 500 вызывается сбоем базы, а не кнопкой. Для SQLite нужно запустить сервер командой `npm start`, затем во втором окне выполнить `npx sequelize-cli db:migrate:undo:all`. Таблицы исчезают под работающим сервером, и первый же запрос не может записать строку в журнал `RequestLogs`. Клиент получает 500 с телом `{ "error": "Internal server error" }`, а подробности остаются только в консоли сервера. Для PostgreSQL достаточно остановить службу базы или контейнер: запросы дают 500, а `GET /health` отвечает 503. Работу восстанавливают команды `npm run`

`db:migrate` и `npm run db:seed`. В автоматическом тесте тот же сбой воспроизведён удалением таблицы `Diagrams` при работающем сервере.

Миграции откатаны командой `npx sequelize-cli db:migrate:undo:all` под работающим сервером, и первый же запрос `http://localhost:3000/diagrams` получает ответ 500, как на рисунке 11.



Рисунок 11 – Ответ GET /diagrams после настоящего сбоя базы: JSON с общим сообщением без подробностей (код 500)

Подробности сбоя остаются только в консоли сервера и показаны на рисунке 12.

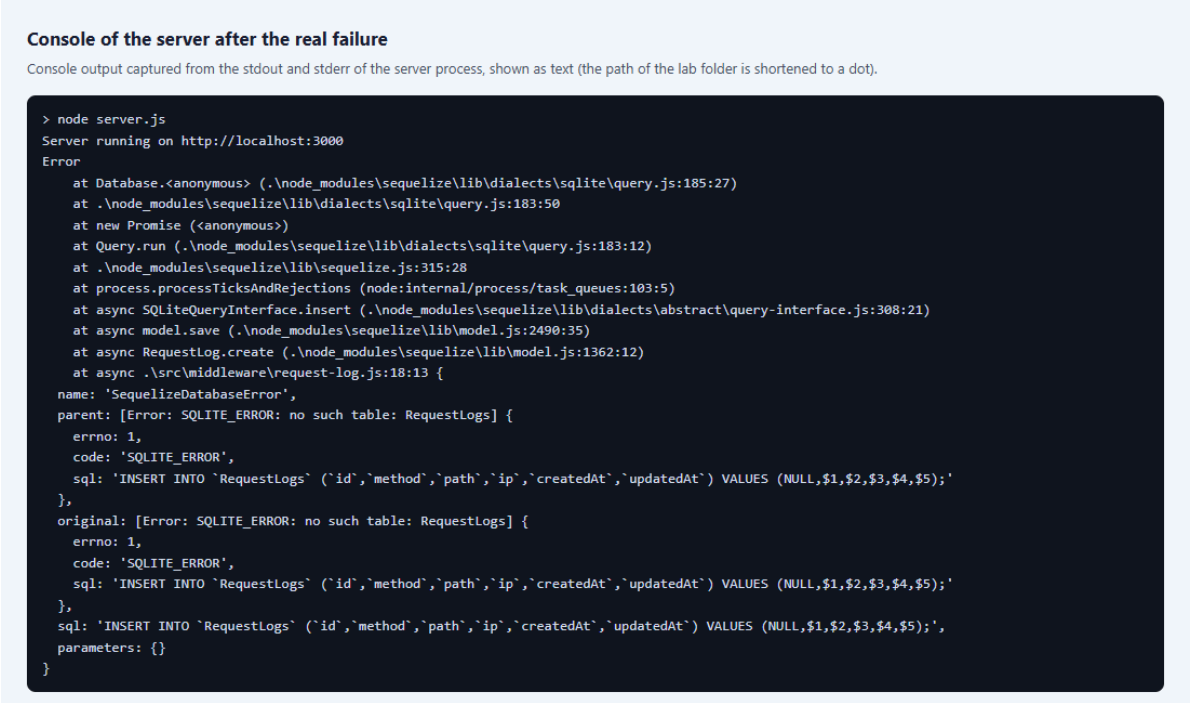


Рисунок 12 – Консоль сервера после сбоя: ошибка отсутствия таблицы RequestLogs (вывод процесса, снятый как текст)

3.10 Ответы на контрольные вопросы

9. Как ограничить количество одновременно действующих сессий для одного пользователя?

JWT без состояния сервер не помнит, поэтому нужна таблица сессий. При входе в неё пишется строка с идентификатором пользователя и уникальным номером токена `jti`, а сам номер кладётся в токен. Если у пользователя уже три активные сессии, самая старая удаляется или вход отклоняется. Middleware проверки токена дополнительно ищет `jti` в таблице, а выход и смена пароля удаляют строки. Более простой вариант это счётчик `tokenVersion` у пользователя: он входит в токен, и его увеличение отзывает все токены сразу. В этой работе ограничение сессий не реализовано, токен действует до истечения срока, поэтому срок выбран коротким.

10. Как защититься от атак по подбору пароля (brute force) на уровне API?

Защита идёт слоями. Ограничитель частоты пропускает не более 20 запросов входа с одного адреса за 15 минут и не более 10 регистраций в час, и при превышении отвечает 429 с заголовком `Retry-After`. После 5 неудачных попыток для одного email вход блокируется на заданный срок, настройка `loginLockMs`. Ответ на неверный пароль и на неизвестный email одинаков, поэтому нельзя узнать, какие адреса зарегистрированы. Пароли хранятся как хеши bcrypt, которые медленно вычисляются, а требования к длине пароля сужают пространство перебора. Счётчики лежат в памяти процесса, при нескольких экземплярах сервера их переносят в общее хранилище, например Redis. Дополнительно применяют капчу и двухфакторный вход.

11. Что произойдёт, если скомпрометирован секретный ключ JWT?

Тот, кто знает ключ, подпишет токен с любым пользователем и любой ролью, в том числе администратора, и сервер примет его как настоящий. Токены нельзя отличить по подписи, поэтому единственное средство это смена ключа: все выданные токены сразу перестают проходить проверку, и пользователи входят заново. В работе ключ берётся из переменной окружения, не хранится в репозитории, а слишком короткий ключ отклоняется при запуске. Ущерб ограничивает короткий срок жизни токена, всего один час. Дополнительную защиту дают ключи с идентификатором `kid` для плавной смены и алгоритм RS256, где подписывает закрытый ключ, а проверяет открытый, поэтому служба, которая только проверяет, подделать токен не может.

3.11 Архитектура и компромиссы

К слоям второй лабораторной добавлен слой безопасности: проверка токена, проверка роли и ограничители частоты стоят отдельными промежуточными обработчиками перед контроллерами. Контроллеры не знают, как проверяется токен, они получают готового пользователя в `req.user`. Это следует правилу зависимостей из «Чистой архитектуры» Мартина: детали, например библиотека JWT, лежат во внешнем слое, а бизнес-правило «менять запись может владелец или администратор» записано одной функцией `mayChange` и проверяется простым тестом.

Главный компромисс связан с состоянием. Токен JWT не хранится на сервере, это упрощает масштабирование, но токен нельзя отозвать до истечения срока. Поэтому срок выбран коротким, а роль берётся из базы при каждом запросе, и удалённый пользователь теряет доступ сразу. Счётчики попыток входа лежат в памяти процесса. Это быстро и просто, но при нескольких экземплярах сервера счётчики не общие. Решение зафиксировано в отчёте, а для боевой системы счётчики переносят в общее хранилище. По Фаулеру («Шаблоны корпоративных приложений») это обычная цена выбора между простотой и распределённым состоянием.

Источники: Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. СПб.: Питер, 2018. Фаулер М. Шаблоны корпоративных приложений. М.: Вильямс, 2016. Фаулер М. Рефакторинг. Улучшение существующего кода. 2-е изд. СПб.: Питер, 2019.

3.12 База данных

Основной базой выбран PostgreSQL. Локальный сервер поднимается командой `docker compose up -d db`, строка подключения лежит в переменной `DATABASE_URL` файла `.env`. Файл SQLite нужен только автоматическим тестам. Миграции, сид и сервер проверены на настоящем PostgreSQL 16 в контейнере: регистрация, вход, создание диаграммы, фильтр по нотации и поиск по названию работают так же, как на SQLite, а чужому пользователю список пользователей отвечает 403.

ЗАКЛЮЧЕНИЕ

В проект DSLKit добавлены пользователи с паролями в виде хешей bcrypt, регистрация, вход с токеном JWT, промежуточный обработчик проверки токена и защищённые маршруты, включая профиль текущего пользователя. Дополнительный механизм ролевой модели разделяет права администратора, владельца диаграммы и постороннего пользователя. Тесты подтверждают как успешные сценарии, так и отказы на подделанных, просроченных и чужих токенах. К методам добавлен частичный `PATCH` с теми же правами, что у `PUT`. Остаётся проверка на настоящем PostgreSQL. Запросы подтверждены снимками браузера и записью обмена, снятой скриптом.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг файла user.js

```
const { DataTypes } = require('sequelize');

const ROLES = ['user', 'admin'];

const defineUser = (sequelize) => {
  const User = sequelize.define('User', {
    // Lowercased on the way in, the database also checks it, so
one address is one account.
    email: {
      type: DataTypes.STRING(254),
      allowNull: false,
      unique: true,
      set(value) {
        this.setDataValue('email', typeof value === 'string' ? v
alue.trim().toLowerCase() : value);
      },
    },
    passwordHash: { type: DataTypes.STRING(100), allowNull: fals
e },
    role: { type: DataTypes.STRING(10), allowNull: false, defaul
tValue: 'user', validate: { isIn: [ROLES] } },
  });

  // The hash must never leave the server, so the public view is
built in one place.
  User.prototype.toPublic = function toPublic() {
    return { id: this.id, email: this.email, role: this.role, cr
eatedAt: this.createdAt };
  };
  // Whatever serializes a User by accident (res.json(user), log
s, JSON.stringify) still cannot show the hash.
  User.prototype.toJSON = function toJSON() {
    return this.toPublic();
  };
  return User;
};

module.exports = { defineUser, ROLES };
```

ПРИЛОЖЕНИЕ Б

(обязательное)

Листинг миграции создания таблицы Users

```
// A migration is a snapshot, so the allowed values are copied h
ere instead of imported.
const ROLES = ['user', 'admin'];

module.exports = {
  async up(queryInterface, Sequelize) {
    const { fn, col, where, Op } = Sequelize;
    await queryInterface.createTable('Users', {
      id: { allowNull: false, autoIncrement: true, primaryKey: t
rue, type: Sequelize.INTEGER },
      email: { allowNull: false, unique: true, type: Sequelize.S
TRING(254) },
      // A bcrypt hash is 60 characters; 100 leaves room for a s
tronger variant.
      passwordHash: { allowNull: false, type: Sequelize.STRING(1
00) },
      role: { allowNull: false, type: Sequelize.STRING(10), defa
ultValue: 'user' },
      createdAt: { allowNull: false, type: Sequelize.DATE },
      updatedAt: { allowNull: false, type: Sequelize.DATE },
    });
    await queryInterface.addConstraint('Users', {
      type: 'check',
      fields: ['role'],
      name: 'users_role_check',
      where: { role: { [Op.in]: ROLES } },
    });
    // Emails are stored lowercased, so the unique index cannot
be dodged with another letter case.
    await queryInterface.addConstraint('Users', {
      type: 'check',
      fields: ['email'],
      name: 'users_email_lowercase_check',
      where: where(fn('lower', col('email')), col('email')),
    });
    // SQLite does not enforce VARCHAR length, so the length and
the empty value are checked explicitly.
    await queryInterface.addConstraint('Users', {
      type: 'check',
      fields: ['email'],
      name: 'users_email_length',
      where: where(fn('length', col('email')), { [Op.between]:
[3, 254] }),
    });
    await queryInterface.addConstraint('Users', {
      type: 'check',
```

```
      fields: ['passwordHash'],
      name: 'users_password_hash_not_empty',
      where: where(fn('length', col('passwordHash')), { [Op.gt]:
0 }),
    });
  },

  async down(queryInterface) {
    await queryInterface.dropTable('Users');
  },
};
```

ПРИЛОЖЕНИЕ В
(обязательное)
Листинг файла auth.js

```
const jwt = require('jsonwebtoken');
const { unauthorized, forbidden } = require('../errors');
const { ISSUER } = require('../settings');

const bearerToken = (req) => {
  const [scheme, token, ...rest] = (req.get('Authorization') ||
'').split(' ');
  return scheme === 'Bearer' && token && rest.length === 0 ? tok
en : undefined;
};

// Verifies the Bearer token, then loads the user so a deleted a
ccount or a changed role takes effect at once.
// req.user holds { id, email, role } taken from the database, n
ot from the token.
const createAuthenticate =
  ({ User }, { jwtSecret }) =>
  async (req, _res, next) => {
    const token = bearerToken(req);
    if (!token) return next(unauthorized('Missing Bearer toke
n'));
    let payload;
    try {
      payload = jwt.verify(token, jwtSecret, { algorithms: ['HS2
56'], issuer: ISSUER });
    } catch (error) {
      return next(unauthorized(error.name === 'TokenExpiredErro
r' ? 'Token expired' : 'Invalid token'));
    }
    // A signed token without a usable integer id cannot name a
user; it is refused like any other bad token.
    if (!Number.isSafeInteger(payload.id)) return next(unauthori
zed('Invalid token'));
    const user = await User.findByPk(payload.id);
    if (!user) return next(unauthorized('User no longer exist
s'));
    req.user = { id: user.id, email: user.email, role: user.role
};
    return next();
  };

const requireRole = (role) => (req, _res, next) => (req.user.rol
e === role ? next() : next(forbidden(`Requires the ${role} role
`)));

module.exports = { createAuthenticate, requireRole };
```

ПРИЛОЖЕНИЕ Г

(обязательное)

Листинг файла auth.controller.js

```
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const { unauthorized, conflict, tooManyRequests } = require('../
errors');
const { ISSUER } = require('../settings');

const createAuthController = ({ User }, { jwtSecret, jwtExpiresI
n, bcryptRounds }, loginGuard) => {
  // Compared when the email is unknown, so a missing account co
sts the same time as a wrong password.
  const dummyHash = bcrypt.hashSync('not-a-real-password', bcryp
tRounds);

  return {
    register: async (req, res, next) => {
      const { email, password } = req.credentials;
      const passwordHash = await bcrypt.hash(password, bcryptRou
nds);
      try {
        const user = await User.create({ email, passwordHash });
        res.status(201).location('/profile').json(user.toPublic
());
      } catch (error) {
        if (error.name !== 'SequelizeUniqueConstraintError') thr
ow error;
        next(conflict('A user with this email already exists'));
      }
    },

    login: async (req, res, next) => {
      const { email, password } = req.credentials;
      // A locked email is refused before the password is looked
at, so guessing cannot continue and the answer does not depend o
n the account.
      // The counter is kept per address and email: someone else
where cannot lock the account of the real owner.
      const guardKey = `${req.ip}|${email}`;
      const wait = loginGuard.retryAfter(guardKey);
      if (wait > 0) return next(tooManyRequests('Too many failed
logins, try again later', wait));
      const user = await User.findOne({ where: { email } });
      const matches = await bcrypt.compare(password, user ? use
r.passwordHash : dummyHash);
      if (!user || !matches) {
        loginGuard.fail(guardKey);
        return next(unauthorized('Invalid email or password'));
```

```
      }
      loginGuard.succeed(guardKey);
      // The role in the token is only a hint for the client: th
e server asks the database on every request.
      const token = jwt.sign({ id: user.id, email: user.email, r
ole: user.role }, jwtSecret, {
        algorithm: 'HS256',
        expiresIn: jwtExpiresIn,
        issuer: ISSUER,
        subject: String(user.id),
      });
      return res.json({ token, tokenType: 'Bearer', expiresIn: j
wtExpiresIn });
    },
  };
};

module.exports = { createAuthController };
```

ПРИЛОЖЕНИЕ Д
(обязательное)
Листинг файла validate-auth.js

```
const { badRequest, unauthorized } = require('../errors');
const { ROLES } = require('../models/user');
const LIMITS = require('../../limits.json');

const EMAIL = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const MIN_PASSWORD = LIMITS.passwordMin;
// bcrypt ignores everything after the 72nd byte, so longer pass
words would silently weaken.
const MAX_PASSWORD_BYTES = LIMITS.passwordMaxBytes;

const objectBody = (body) => body !== null && typeof body === 'o
bject' && !Array.isArray(body);
const normalizeEmail = (email) => (typeof email === 'string' ? e
mail.trim().toLowerCase() : email);

// Control characters (NUL, tab, escape) are not part of an addr
ess; NUL also breaks the SQL text of a query.
const CONTROL = /[\u0000-\u001F\u007F]/;

const validEmail = (email) => typeof email === 'string' && emai
l.length <= LIMITS.email && EMAIL.test(email) && !CONTROL.test(e
mail);

const passwordProblem = (password) => {
  if (typeof password !== 'string') return 'must be a string';
  if (password.trim() === '') return 'must not be empty or only
spaces';
  if (password.length < MIN_PASSWORD) return `must be at least
${MIN_PASSWORD} characters`;
  if (Buffer.byteLength(password) > MAX_PASSWORD_BYTES) return `
must be at most ${MAX_PASSWORD_BYTES} bytes`;
  return undefined;
};

const reject = (next, details) => next(badRequest('Invalid crede
ntials data', details));

const validateRegistration = (req, _res, next) => {
  if (!objectBody(req.body)) return next(badRequest('Request bod
y must be a JSON object'));
  const email = normalizeEmail(req.body.email);
  const details = [];
  if (!validEmail(email)) details.push({ field: 'email', messag
e: 'must be a valid email address' });
  const problem = passwordProblem(req.body.password);
  if (problem) details.push({ field: 'password', message: proble
```

```
m });
  if (details.length > 0) return reject(next, details);
  req.credentials = { email, password: req.body.password };
  return next();
};

// Login only needs the two values to exist; the rules above wou
ld leak which part of a wrong password is fine.
const validateLogin = (req, _res, next) => {
  if (!objectBody(req.body)) return next(badRequest('Request bod
y must be a JSON object'));
  const { email, password } = req.body;
  if (typeof email !== 'string' || typeof password !== 'string'
|| !email.trim() || !password.trim()) {
    return reject(next, [{ field: 'email, password', message: 'b
oth are required strings' }]);
  }
  // A value longer than any stored one cannot match an account,
so it gets the usual answer without hashing it.
  if (email.length > LIMITS.email || CONTROL.test(email) || Buff
er.byteLength(password) > MAX_PASSWORD_BYTES) {
    return next(unauthorized('Invalid email or password'));
  }
  req.credentials = { email: normalizeEmail(email), password };
  return next();
};

const validateRole = (req, _res, next) => {
  const role = objectBody(req.body) ? req.body.role : undefined;
  if (!ROLES.includes(role)) return next(badRequest('Invalid rol
e data', [{ field: 'role', message: `must be one of ${ROLES.join
(', ')}` }]));
  req.newRole = role;
  return next();
};

module.exports = { validateRegistration, validateLogin, validate
Role };
```

ПРИЛОЖЕНИЕ Е
(обязательное)
Листинг файла users.routes.js

```
const { Router } = require('express');
const { requireRole } = require('../middleware/auth');
const { parseId } = require('../middleware/validate');
const { validateRole } = require('../middleware/validate-auth');
const { allowMethods } = require('../middleware/http');

// Every route here needs a valid token; the /users routes also
need the admin role.
// The 405 and OPTIONS answers come without a token, because the
y say nothing about the data.
const createUsersRouter = (controller, authenticate) => {
  const router = Router();
  const admin = [authenticate, requireRole('admin')];
  router
    .route('/profile')
    .get(authenticate, controller.profile)
    .all(allowMethods(['GET']));
  router
    .route('/users')
    .get(...admin, controller.list)
    .all(allowMethods(['GET']));
  router
    .route('/users/:id/role')
    .patch(...admin, parseId('userId', 'User'), validateRole, co
ntroller.setRole)
    .all(allowMethods(['PATCH']));
  router
    .route('/users/:id')
    .delete(...admin, parseId('userId', 'User'), controller.remo
ve)
    .all(allowMethods(['DELETE']));
  return router;
};

module.exports = { createUsersRouter };
```

ПРИЛОЖЕНИЕ Ж

(обязательное)

Листинг файла diagrams.controller.js

```
const { notFound, forbidden } = require('../errors');
const { Op, fn, col, where } = require('sequelize');
const { parseListQuery } = require('../middleware/list-query');
const { NOTATIONS, LIMITS } = require('../models/diagram.constan
ts');

const missing = (id) => notFound(`Diagram ${id} not found`);

const SORTABLE = ['id', 'title', 'notation', 'priority', 'ownerI
d', 'createdAt'];

// A part of the title is searched with instr (SQLite) or strpos
(PostgreSQL) on the lowercased title, not with LIKE,
// so the characters % and _ in the search text are plain charac
ters and need no escaping.
const filterFor = (dialect, { notation, q }) => {
  const conditions = [];
  if (notation !== undefined) conditions.push({ notation });
  if (q !== undefined) conditions.push(where(fn(dialect === 'sql
ite' ? 'instr' : 'strpos', fn('lower', col('title')), q), { [Op.
gt]: 0 }));
  return conditions.length > 0 ? { [Op.and]: conditions } : {};
};

// Only the owner or an admin may change or delete a diagram.
const mayChange = (user, diagram) => user.role === 'admin' || di
agram.ownerId === user.id;

// Express 5 forwards a rejected promise to the error handler, s
o no try/catch is needed here.
const createDiagramsController = ({ Diagram }) => ({
  // ?limit=&offset=&sort=&order= (see list-query.js) page and s
ort the result, the answer stays an array and X-Total-Count hold
s the full size.
  list: async (req, res) => {
    const { limit, offset, sort, order, notation, q } = parseLis
tQuery(req.query, SORTABLE, { notations: NOTATIONS, maxQuery: LI
MITS.title });
    const { count, rows } = await Diagram.findAndCountAll({
      where: filterFor(Diagram.sequelize.getDialect(), { notatio
n, q }),
      order: [
        [sort, order.toUpperCase()],
        ['id', 'ASC'],
      ],
      limit,
```

```
      offset,
    });
    res.set('X-Total-Count', String(count));
    res.json(rows);
  },

  get: async (req, res, next) => {
    const diagram = await Diagram.findByPk(req.diagramId);
    return diagram ? res.json(diagram) : next(missing(req.diagra
mId));
  },

  create: async (req, res) => {
    const diagram = await Diagram.create({ ...req.diagramFields,
ownerId: req.user.id });
    res.status(201).location(`/diagrams/${diagram.id}`).json(dia
gram);
  },

  replace: async (req, res, next) => {
    const diagram = await Diagram.findByPk(req.diagramId);
    if (!diagram) return next(missing(req.diagramId));
    if (!mayChange(req.user, diagram)) return next(forbidden('On
ly the owner or an admin can change this diagram'));
    return res.json(await diagram.update(req.diagramFields));
  },

  // PATCH: only the sent fields change, the same owner-or-admin
rule as PUT applies.
  update: async (req, res, next) => {
    const diagram = await Diagram.findByPk(req.diagramId);
    if (!diagram) return next(missing(req.diagramId));
    if (!mayChange(req.user, diagram)) return next(forbidden('On
ly the owner or an admin can change this diagram'));
    return res.json(await diagram.update(req.diagramFields));
  },

  remove: async (req, res, next) => {
    const diagram = await Diagram.findByPk(req.diagramId);
    if (!diagram) return next(missing(req.diagramId));
    if (!mayChange(req.user, diagram)) return next(forbidden('On
ly the owner or an admin can delete this diagram'));
    await diagram.destroy();
    return res.status(204).end();
  },
});

module.exports = { createDiagramsController };
```

ПРИЛОЖЕНИЕ И
(обязательное)
Листинг файла list-query.js

```
const { badRequest } = require('../errors');

// Limits of the list query. The page size is capped so one requ
est cannot ask for the whole table,
// and the numbers are read as short digit strings, so 1e9, -1,
0x10 or a huge value are refused with 400.
const MAX_LIMIT = 100;
const MAX_OFFSET = 1000000;
const ORDERS = ['asc', 'desc'];
// A control character such as NUL cannot be in a title, and som
e databases fail on it inside a query text, so the search refuse
s it.
const CONTROL = /[\u0000-\u001f\u007f-\u009f]/;

// Reads ?limit=, ?offset=, ?sort=, ?order=, ?notation= and ?q=
from req.query.
// sortable: the field names a client may sort by (a whitelist,
so a field name never reaches the database unchecked).
// Without limit the whole list is returned (the clients of the
earlier labs expect that), without sort the order is by id.
// A bad value throws the 400 error with the list of problems, E
xpress 5 passes it to the error handler.
// filters: { notations: allowed values of ?notation=, maxQuery:
longest ?q= }. ?q= is a case-insensitive part of the title.
const parseListQuery = (query, sortable, filters = {}) => {
  const problems = [];
  const integer = (name, min, max) => {
    const value = query[name];
    if (value === undefined) return undefined;
    if (typeof value !== 'string' || !/^\d{1,9}$/.test(value) ||
Number(value) < min || Number(value) > max) {
      problems.push({ field: name, message: `${name} must be an
integer from ${min} to ${max}` });
      return undefined;
    }
    return Number(value);
  };
  const limit = integer('limit', 1, MAX_LIMIT);
  const offset = integer('offset', 0, MAX_OFFSET) ?? 0;

  const sort = query.sort ?? 'id';
  if (typeof sort !== 'string' || !sortable.includes(sort)) {
    problems.push({ field: 'sort', message: `sort must be one o
f: ${sortable.join(', ')}` });
  }
  const order = typeof query.order === 'string' ? query.order.to
```

```
LowerCase() : (query.order ?? 'asc');
  if (!ORDERS.includes(order)) problems.push({ field: 'order', m
essage: 'order must be asc or desc' });

  const { notations = [], maxQuery = 80 } = filters;
  let notation;
  if (query.notation !== undefined) {
    if (typeof query.notation !== 'string' || !notations.include
s(query.notation)) problems.push({ field: 'notation', message: `
notation must be one of: ${notations.join(', ')}` });
    else notation = query.notation;
  }
  let q;
  if (query.q !== undefined) {
    const text = typeof query.q === 'string' ? query.q.trim() :
'';
    if (text === '' || text.length > maxQuery || CONTROL.test(te
xt)) problems.push({ field: 'q', message: `q must be text of 1 t
o ${maxQuery} characters without control characters` });
    else q = text.toLowerCase();
  }

  if (problems.length) throw badRequest('Invalid list query', pr
oblems);
  return { limit, offset, sort, order, notation, q };
};

module.exports = { parseListQuery, MAX_LIMIT, MAX_OFFSET };
```

ПРИЛОЖЕНИЕ К
(обязательное)
Листинг файла validate.js

```
const { badRequest } = require('../errors');
const { NOTATIONS, PRIORITIES, DEFAULT_PRIORITY, LIMITS, MAX_ID
} = require('../models/diagram.constants');

// Spaces of every kind plus zero-width characters (joiners, wor
d joiner, BOM) look empty, so they count as empty.
const INVISIBLE = /[\s\u200B-\u200D\u2060\uFEFF]/g;
// Control characters (NUL, bell, escape) are not text; a single
-line field must not carry them.
const CONTROL = /[\u0000-\u001F\u007F]/;

// Text is checked after trimming, so the stored value is what w
as measured.
const text =
  (name, singleLine = false) =>
  (v) => {
    if (typeof v !== 'string' || v.replace(INVISIBLE, '') ===
'') return 'must be a non-empty string';
    if (singleLine && CONTROL.test(v)) return 'must not contain
control characters';
    return v.trim().length <= LIMITS[name] || `must be at most
${LIMITS[name]} characters`;
  };

const checks = {
  title: text('title', true),
  notation: (v) => NOTATIONS.includes(v) || `must be one of ${NO
TATIONS.join(', ')}`,
  sourceCode: text('sourceCode'),
  priority: (v) => PRIORITIES.includes(v) || `must be one of ${P
RIORITIES.join(', ')}`,
};

// Builds the id check for a route param. The raw text is checke
d first because Number() would accept '0x1', '1e0' and ' 1'.
const parseId =
  (key = 'diagramId', label = 'Diagram') =>
  (req, _res, next) => {
    const id = Number(req.params.id);
    // Ids above the INTEGER column range would make PostgreSQL
fail with a 500, so they are rejected here.
    if (!/^[1-9]\d*$/.test(req.params.id) || id > MAX_ID) return
next(badRequest(`${label} id must be a positive integer up to
${MAX_ID}`));
    req[key] = id;
    return next();
```

```
  };

// POST and PUT both carry the whole diagram, so one validator s
erves both.
// The owner is never taken from the body: the controller sets i
t from the token. The title is trimmed before saving.
// Priority is optional and falls back to the default; unknown f
ields are dropped.
const isObject = (body) => body !== null && typeof body === 'obj
ect' && !Array.isArray(body);

const validateDiagram = (req, _res, next) => {
  const body = req.body;
  if (!isObject(body)) {
    return next(badRequest('Request body must be a JSON objec
t'));
  }
  const fields = { ...body, priority: body.priority ?? DEFAULT_P
RIORITY };
  const details = Object.entries(checks)
    .map(([name, check]) => ({ field: name, result: check(fields
[name]) }))
    .filter(({ result }) => result !== true)
    .map(({ field, result }) => ({ field, message: result }));
  if (details.length > 0) return next(badRequest('Invalid diagra
m data', details));
  req.diagramFields = Object.fromEntries(Object.keys(checks).map
((name) => [name, name === 'title' ? fields[name].trim() : field
s[name]]));
  return next();
};

// PATCH carries only the fields to change. Each sent field gets
the same check as in POST and PUT,
// missing fields are left alone, and a body without a single kn
own field is refused. The owner cannot be sent at all.
const validatePartial = (req, _res, next) => {
  const body = req.body;
  if (!isObject(body)) return next(badRequest('Request body must
be a JSON object'));
  const names = Object.keys(checks).filter((name) => Object.hasO
wn(body, name));
  if (names.length === 0) return next(badRequest(`Send at least
one of: ${Object.keys(checks).join(', ')}`));
  const details = names
    .map((name) => ({ field: name, result: checks[name](body[nam
e]) }))
    .filter(({ result }) => result !== true)
    .map(({ field, result }) => ({ field, message: result }));
  if (details.length > 0) return next(badRequest('Invalid diagra
m data', details));
  req.diagramFields = Object.fromEntries(names.map((name) => [na
```

```
me, name === 'title' ? body[name].trim() : body[name]]));
  return next();
};

module.exports = { parseId, validateDiagram, validatePartial };
```